



Universidad
Indoamérica

ユニット 2・オペレーティングシステムの基礎

システムコールと低水準入出力

Universidad Tecnológica Indoamérica 演習 TD1

抽象化

- 単純化と標準化
- カーネルとプログラムの移植性 (POSIX)
- ドライバ開発を容易にする
- 実行環境を安定させる

このコースで扱う抽象化

ファイル (U3)・プロセスと仮想メモリ (U4)・通信 (U5、U6、U8)・スレッド (U7)・そしてエージェント (U9)

保護

- 識別と利用・アクセス権
- プロセス間・ユーザー間の隔離
- データの暗号化
- セキュリティポリシー（ネットワーク、ファイル...）

2026 年：同じ原理が新しい「ユーザー」にも当てはまります：**AI エージェント**（独自の ID、最小権限、監査 - U9）。

仮想化

移植性とプログラミング

- VFS : ext4、Btrfs、NFS、`/proc ...` に共通のインタフェース
- ハードウェアを均質に見せる (USB、SATA、NVMe のディスクを同じように扱う)
- POSIX がシステム関数を標準化する

安定性と進化

- プロセスの隔離 : 1 つの障害でマシン全体が止まらない
- マシンや別の OS をシミュレートする : QEMU/KVM、WSL 2
- コンテナ : 名前空間 + cgroups
- IPv6 上の IPv4

プロセスという考え方

- 各プロセスは自分の
コンテキストを持つ
- メモリの**私的な**視点：他の
プロセスのメモリは見えない
- 隔離されていて**自分だけだと思っ**
ている：他のプロセスも
システム自体も隠されている



プロセッサの2つの実行モード

ユーザーモード（非特権）

既定のモード。プログラムは**自分のメモリの中で**計算し、読み書きするだけです。

カーネルモード（特権）

それ以外のことは、特別な命令（システムコール）でプロセスが**許可を求めます**：ファイルを開く・変更する・名前を変える、メモリを確保する、タスクを作る・終える...

ハードウェアがモードを切り替え、OS が操作を許可してよいかを確認します。他のタスクやユーザーに害を及ぼしうるものすべてが対象です。

カーネルモードとカーネル

- 一部の命令（メモリ管理、割り込み）はユーザーモードでは**禁止**されている
- カーネルはプロセスを**管理する**が、**プロセスではない**
- 資源とハードウェアへのアクセスを管理し、常にメモリに常駐し、起動時に読み込まれる（UI、A1.3）
- ユーザープロセスはカーネルのサービスを使うために**カーネルモードに切り替わる**

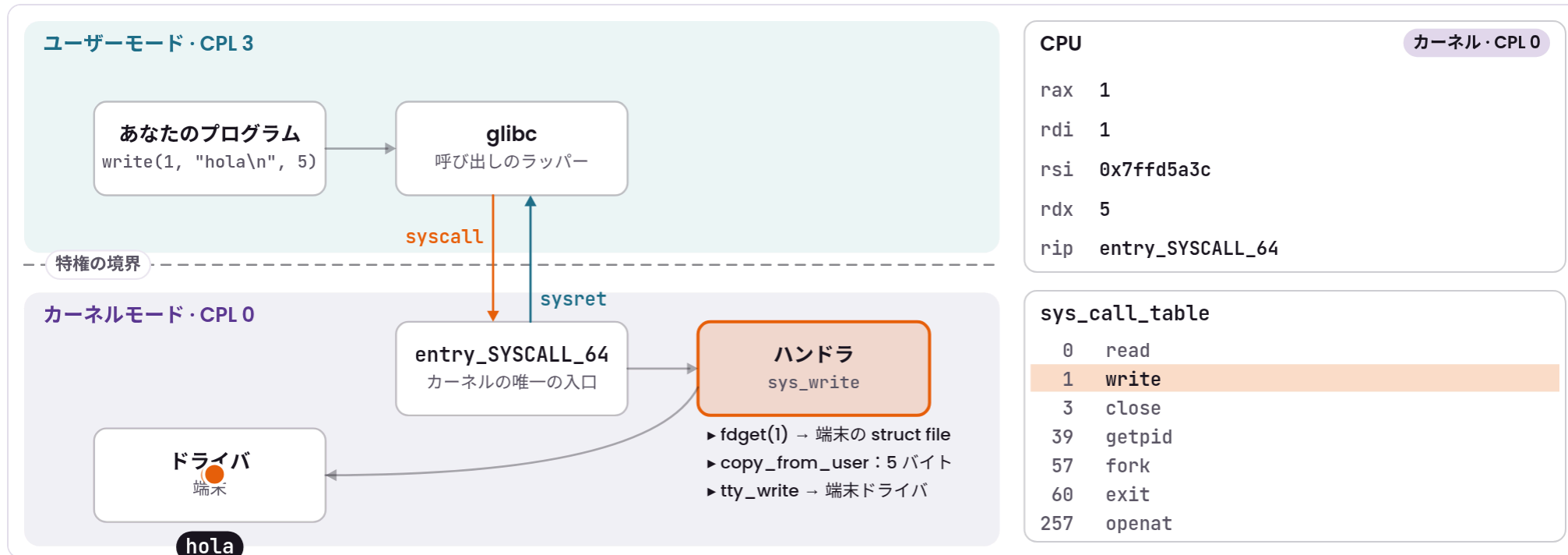
システムコール：インタフェース

- ユーザーモードとカーネルモードの間のインタフェース
- モード切り替えは命令で起こる：`ta` (Sparc)、`int 0x80` (旧 x86)、現在は `syscall` (x86-64)、`svc #0` (ARM64)、`ecall` (RISC-V)
- カーネルには入口のハンドラとシステムコール表がある

システムコールは高くつく！

元資料では「数百サイクル」とありました。コースのマシンでの実測：
`read` + `write` 1回あたり約 **0.7 μ s** (数千サイクル)。

write(1, "hola\n", 5) の旅



objdump で見る命令

レジスタ	WRITE(1, BUF, 5)
rax	1 (write の番号)
rdi	1 (ディスクリプタ)
rsi	buf のアドレス
rdx	5

実際の出力

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-  
gnu/libc.so.6 | grep -A6 -E '^[0-9a-f]+'  
<__write_nocancel@@'  
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:  
129060:    endbr64  
129064:    mov     $0x1,%eax  
129069:    syscall  
12906b:    cmp     $0xffffffffffffffff,000,%rax  
129071:    ja      129078  
<__write_nocancel@@GLIBC_PRIVATE+0x18>  
129073:    ret
```

エラー：-1、`errno`、`perror`

- 失敗した呼び出しは `-1` を返し、コードを `errno` に残す
- メッセージ： `void perror(const char *s);`
または `fprintf` の `%m` (glibc の拡張)
- このコースでは： `exit_if(rc == -1, "write")`

実際の出力

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

c

```
ssize_t rc = write(STDOUT_FILENO, buf, n);
if (rc == -1)
    fprintf(stderr, "write: %m\n");
exit_if(rc == -1, "write");
```

呼び出しを追跡する： **strace**

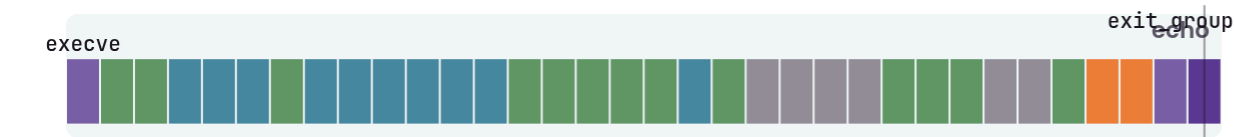
実際の出力

```
$ strace -r ./echo hola > /dev/null      # the whole life of the process
0.000000 execve("./echo", [ "./echo", "hola"], 0x7ffff5a76120 /* 129 vars */) = 0
0.000965 brk(NULL)                      = 0x615cdaf80000
0.000267 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x747beb566000
0.000354 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
0.000362 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
0.000116 fstat(3, {st_mode=S_IFREG|0644, st_size=108847, ...}) = 0
0.000149 mmap(NULL, 108847, PROT_READ, MAP_PRIVATE, 3, 0) = 0x747beb54b000
0.000156 close(3)                       = 0
...
```

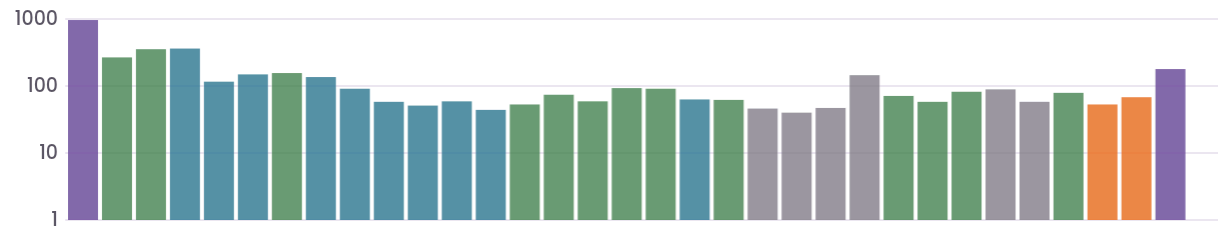
実際の出力

```
0.000089 getRandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_NONBLOCK) = 8
0.000058 munmap(0x747beb54b000, 108847) = 0
0.000079 write(1, "hola", 4)              = 4
0.000053 write(1, "\n", 1)                = 1
0.000068 exit_group(0)                  = ?
0.000179 +++ exited with 0 +++
```

トレース全体を1呼び出しずつ



次の呼び出しまでの時間 (μs、対数)



■ プロセス 3 ■ メモリ 13 ■ ファイルとディスクリプタ 10 ■ 出力 (write) 2 ■ その他 6

```
set_tid_address(0x747beb548d68) = 809641
set_robust_list(0x747beb548a20, 24) = 0
rseq(0x747beb548680, 0x21, 0, 0x53053053) = 0
mprotect(0x747beb40e000, 16384, PROT_READ) = 0
mprotect(0x615caa96a000, 4096, PROT_READ) = 0
mprotect(0x747beb5ab000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, ...
getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_...
munmap(0x747beb54b000, 108847) = 0
write(1, "hola", 4)      = 4
write(1, "\n", 1)       = 1
exit_group(0)           = ?
+++ exited with 0 +++
```

システムコールとライブラリ関数

システムコール (man 2)

inode: `stat`、`unlink`、`chmod`、`link` ...
ディスクリプタ: `open`、`read`、`write`、
`lseek`、`close`、`fcntl` ...

C ライブラリ (man 3)

`fopen`、`fread`、`fprintf`、`fclose` ... は
システムコールではありません：
ディスクリプタの代わりに（バッファ付きの）
`FILE *` を使います。 `fileno()` で相互に
変換できます。

データを読む：read

C

```
ssize_t read(int fd, void *buf, size_t count);
```

- `fd` のファイルから**最大** `count` バイトを、確保済みの `buf` に読み込む（`count` ≤ そのサイズ）
- 読んだバイト数、**ファイル終端なら 0**、エラーなら -1 を返す
- 要求より**少ないこともある**：パイプ、端末、ネットワーク、シグナル（何も読まなければ `EINTR`）

データを書く：write

C

```
ssize_t write(int fd, const void *buffer, size_t count);
```

- `buffer` の先頭 `count` バイトを `fd` のファイルに書く
- エラーなら `-1`、そうでなければ書いたバイト数を返す

元資料の修正：「`count` より少なく書いたらエラー」は常に正しいわけではありません。パイプやソケットが満杯のときや途中でシグナルが来たとき、書き込みは**部分的でも正当**です：残りで繰り返します（`write_all`）。

バグはどこ？

C

```
/* slide 51 of the original */
while ((byte = read(fd, buf, length))  $\neq$  0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;      /* BUG */
}
```

C

```
/* fixed: advance by what was READ */
while (length > 0) {
    byte = read(fd, buf, length);
    if (byte == -1 && errno == EINTR)
        continue;
    exit_if(byte == -1, "read");
    if (byte == 0)           /* end of file */
        break;
    length -= byte;
    buf += byte;
}
```

同じループをファイルとパイプで

元のコード (スライド 51)

```
while ((byte = read(fd, buf, length)) != 0) {  
    if (byte == -1) {  
        perror("read() failed"); exit(1);  
    }  
    length = length - byte;  
    buf = buf + length;  
}
```

buf = 43 length = 0

```
$ strace -e trace=read ./prog (pipe)
```

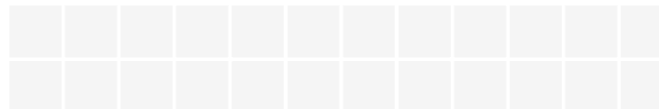
```
read(3, buf+36, 16) = 9
```

```
read(3, buf+43, 7) = 7
```

```
read(3, buf+43, 0) = 0
```

パイプ：書き手が 4 + 4 + 9 + 7 バイトの write() を 4 回

パイプ



プロセスのメモリ：char buf[24] とその後ろ

+0 | r | e | a | d | . | . | . | . | . | . | . | .

+12 | . | . | . | . | . | . | . | . | (|) | m

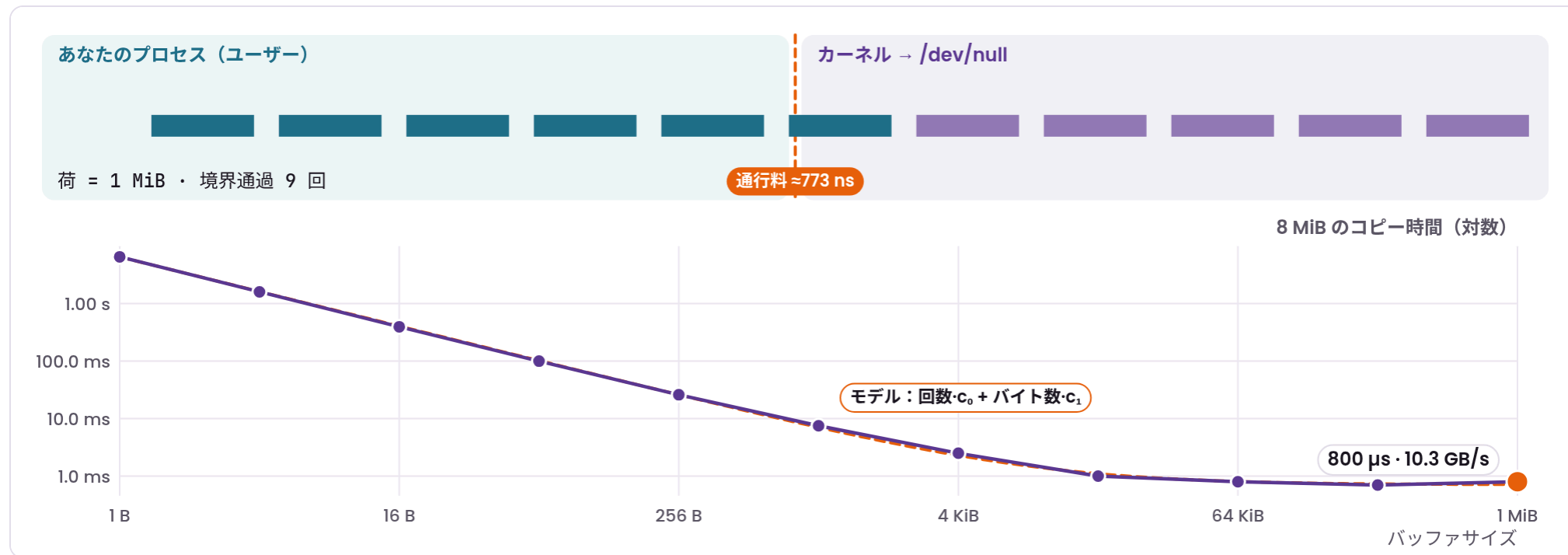
---- buf[24] の終わり・隣のメモリ：他の変数、戻りアドレス...

+24 | ? | ? | ? | ? | ? | ? | ? | ? | ? | ? | ?

+36 | a | y | | r | e | t | u | | l | e | s | s

buf = 43

境界を越えるたびにいくらかかる？実測



境界通過を減らして同じ仕事を

vDSO

カーネルに入らずに
`clock_gettime`。

io_uring

共有キュー：1回の
呼び出しで数百の操作。

eBPF

プロセスを止めずに
システム全体の呼び出しを
観察。

解答付き TD1 + AI チャレンジ

- 7つの演習問題をビルドして実行し、1つをわざと壊す
- `strace ./echo` の各行を説明する
- AI チャレンジ：自分のマシンで呼び出しのコストを測る

Bash

```
make -C tds && tds/tests/run.sh td1
```



Universidad
Indoamérica

まとめ

3つの問い

1. なぜカーネルはプロセスではなく、root でもないのか？
2. ファイル終端で `read` は何を返し、glibc は -9 をどう扱う？
3. スライド 51 の例は何が間違っていた？